CONFIDENTIAL SECURITY DOCUMENT

Mobile Application Penetration Test Report

Example Corp — Example Mobile App (Android/iOS)

ANDROID + iOS

Assessment Type	Black-Box & Grey-Box Mobile Application Penetration Test
App Version(s)	`[FILL: Android v/versionCode; iOS v/build]`
Report Date	1 January 2030
Report Status	**Draft**
Overall Risk Rating	CRITICAL
Classification	**Confidential — Restricted Distribution**

Distribution: This document contains sensitive security information and must not be shared outside authorised personnel responsible for remediation.

Disclaimer: Testing covered the client-supplied application binary/binaries and their backing API(s), on both a jailbroken/rooted device and an emulator, under written authorisation.

1

1 Table of Contents 1

2 Executive Summary 2
2.1 Overview . . . 2
2.2 Overall Risk Rating . . . 2
2.3 Findings by Severity . . . 2

3 Scope & Methodology 3
3.1 In-Scope Assets . . . 3
3.2 Test Devices . . . 3
3.3 Rules of Engagement . . . 3
3.4 Methodology . . . 3
3.5 Severity Definitions . . . 4

4 Application Environment 5

5 Technical Findings 6
5.1 F-01 — [FILL: Short, specific finding title, e.g. "Certificate Pinning Disabled in Production Build"] . . . 6
5.1.1 Description . . . 6
5.1.2 Steps to Reproduce . . . 6
5.1.3 Impact . . . 6
5.1.4 Remediation . . . 6
5.2 F-02 — [FILL: Short, specific finding title, e.g. "Session Token Stored in Cleartext in Shared Preferences"] . . . 7
5.2.1 Description . . . 7
5.2.2 Impact . . . 7
5.2.3 Remediation . . . 7

6 Recommendations 8

7 Appendix A — Tools Used 9

8 Appendix B — Permissions Review 9

2 Executive Summary

Overview

A black-box and grey-box penetration test was conducted against Example Corp — Example Mobile App (Android/iOS) covering the mobile application binary (static analysis), its runtime behaviour on a jailbroken/rooted test device (dynamic analysis), and the backing API(s) it communicates with.

[FILL: One or two sentences on the overall outcome, e.g. "the app stores session tokens in cleartext in shared preferences and disables certificate pinning outside of production builds, allowing full traffic interception."]

CRITICAL — Key Takeaway

Result — [FILL: one-line verdict].

Headline conclusions:

- [FILL: Headline finding 1]
- [FILL: Headline finding 2]
- [FILL: Headline finding 3]

Overall Risk Rating

CRITICAL

Findings by Severity

Severity	Count	Meaning
CRITICAL	[FILL: n]	[FILL: e.g. full account takeover / server compromise from the client alone]
HIGH	[FILL: n]	[FILL: e.g. sensitive data exposure, auth bypass requiring some precondition]
MEDIUM	[FILL: n]	[FILL: e.g. insecure storage of non-critical data, weak transport hardening]
LOW	[FILL: n]	[FILL: e.g. hygiene / defence-in-depth gaps]

3 Scope & Methodology

In-Scope Assets

Asset	Notes
[FILL: Android APK (package name)]	[FILL: Version / build under test]
[FILL: iOS IPA (bundle identifier)]	[FILL: Version / build under test]
[FILL: Backing API(s)]	[FILL: Base URL(s), auth scheme]

Test Devices

Device	Notes
ANDROID	[FILL: Model, OS version, rooted (Magisk/similar)]
iOS	[FILL: Model, iOS version, jailbroken (checkra1n/similar)]

Rules of Engagement

- Testing window: [FILL: start date] – [FILL: end date].
- Test accounts supplied: [FILL: role(s) and count].
- Production data access: [FILL: in / out of scope].

Methodology

Testing followed a methodology aligned with the OWASP Mobile Application Security Testing Guide (MASTG) and OWASP Mobile Top 10:

1. **Static analysis.** Decompilation/disassembly of the binary (`apktool`/`jadx` for Android, `class-dump`/Hopper for iOS) to review hardcoded secrets, insecure storage APIs, and exported/unprotected components.
2. **Dynamic analysis.** Runtime instrumentation (`Frida`/`Objection`) on a rooted/jailbroken device to bypass root/jailbreak and certificate-pinning checks, inspect in-memory data, and hook sensitive method calls.
3. **Network analysis.** Intercepting proxy (`Burp Suite`) traffic capture to assess transport security, certificate validation, and API-level vulnerabilities.
4. **Local storage analysis.** Inspection of the app sandbox / data directory (SQLite databases, shared preferences/plist, keychain/keystore) for sensitive data at rest.

Severity Definitions

Level	Definition
CRITICAL	Remote/unauthenticated compromise of user accounts or backend systems via the client.
HIGH	Sensitive data exposure or auth bypass requiring some local access or precondition.
MEDIUM	Insecure storage/transport of non-critical data; weak hardening controls.
LOW / Info	Hygiene and defence-in-depth gaps.

4 Application Environment

Component	Detail
Client framework	[FILL: e.g. native Kotlin/Swift, React Native, Flutter]
Local storage	[FILL: e.g. SQLite, SharedPreferences/Keychain, Realm]
Authentication	[FILL: e.g. OAuth2 bearer token, biometric-gated session]
Certificate pinning	[FILL: present / absent / bypassable]
Root/jailbreak detection	[FILL: present / absent / bypassable]

5 Technical Findings

F-01 — [FILL: Short, specific finding title, e.g. "Certificate Pinning Disabled in Production Build"]

Finding Metadata			
ID	F-01	**Severity**	CRITICAL
Platform	ANDROID + iOS	**Exploitability**	[FILL: Trivial / Moderate]
Component	[FILL: e.g. network layer / OkHttp client]	**Auth Required**	[FILL: None / Authenticated]

Description

[FILL: Describe the vulnerable component and how it was identified (static disassembly finding, or dynamic Frida-hook observation).]

Steps to Reproduce

1. [FILL: Step 1 - e.g. install a Burp CA certificate and configure the device proxy.]
2. [FILL: Step 2 - e.g. launch the app and observe traffic is intercepted without a pinning error.]

Impact

CRITICAL — Key Takeaway

[FILL: What an attacker in a privileged network position (public Wi-Fi, malicious VPN profile, compromised router) can now do - e.g. read/modify all API traffic including auth tokens.]

Remediation

- [FILL: e.g. implement certificate/public-key pinning consistently across all build variants.]
- [FILL: e.g. fail closed (refuse the connection) rather than fail open when pinning validation errors.]

F-02 — [FILL: Short, specific finding title, e.g. "Session Token Stored in Cleartext in Shared Preferences"]

Finding Metadata			
ID	F-02	**Severity**	MEDIUM
Platform	ANDROID	**Exploitability**	[FILL: Requires local/physical access or malware]
Component	[FILL: e.g. session manager]	**Auth Required**	[FILL: None]

Description

[FILL: Describe the storage location and why it is readable by another app or a root/backup-based extraction.]

Impact

HIGH — Impact Statement

[FILL: What can be done with the extracted token/data - e.g. session hijacking without the victim's password.]

Remediation

- [FILL: e.g. move the token into EncryptedSharedPreferences / Keystore-backed storage.]

6 Recommendations

Priority	Recommendation
Immediate	[FILL: e.g. move tokens out of SharedPreferences/plist into Keystore/Keychain]
Short-term	[FILL: e.g. enforce certificate pinning in all build variants]
Long-term	[FILL: e.g. add root/jailbreak + Frida detection with server-side enforcement]

7 Appendix A — Tools Used

- [FILL: e.g. jadx, apktool, Frida, Objection, Burp Suite]

8 Appendix B — Permissions Review

Permission	Justified?
[FILL: e.g. android.permission.READ_CONTACTS]	[FILL: yes/no - why]